CS315 Project: DB Optimization

- Chinmay Pillai (200298)

Introduction

The project focuses on gaining a better understanding of how PostgreSQL works/optimizes queries, and practically implementing these inferences alongside the material taught in class. This is achieved by:

1. Analysing how PostgreSQL processes queries and indexes.
2. Applying the obtained inferences to improve an existing production database.
3. Designing and implementing a production database to store and retrieve a given set of attributes efficiently based on normalization theory.

Below concepts from the class has been analysed/implemented:

1. Normalization
2. Indexing
3. Query Optimization
4. Query Processing
5. Choosing Optimal Database Implementation

Below concepts beyond the material taught in the class has been explored/implemented:

1. Functional Indexing
2. Query Plan Analysis
3. Postgres Optimizer Analysis
4. Reindexing and Postgres' Autovacuum
5. Bitmap Heap Scan

The project involves 4 sub-projects each involving different databases and requirements.

I. Database Design, Normalization

Normalization

A database needs to be designed for the below attributes:

1. Product Description (prod_desc) (str)
2. Scanned (bool)
3. Weight (float)
4. Current Week Count (count) (int)
5. Weekly EMA of Count (ema) (float)

Functional Dependencies (FD):

1. (Prod_desc) -> (weight, count, ema)
2. (Prod_desc) - > (Scanned)
3. (weight) -> (scanned) (scanned is true if weight is not null, false otherwise)
4. (count, ema) - > (scanned) (scanned is false if count and ema are null, true otherwise)

Note: if scanned is false, weight is null, else count and ema are null. If (weight) is not null, (count, ema) will be null and vice versa, i.e, only one of the 2 sets of attributes will be not null and the other will be null in all cases.

Requirements:

1. High frequency updates to ‘count’ attribute of a specific tuple
2. Quick Full-Text Search of all tuples for closest ‘prod_desc’ and its corresponding Weight
3. Weekly Update of EMA of all tuples
4. Updation of Scanned attribute on scan

Main Requirements:

1. High Frequency update
2. Quick Full-Text Search

Functional Dependency Analysis:

Consider a single database with all the attributes. It is **1NF by default** due to all attributes being atomic. Given prod_desc, all other attributes can be found using the FDs:

- (Prod_desc) -> (weight, count, ema)
- (Prod_desc) - > (Scanned)

Hence, prod_desc is a candidate key. There are no other candidate keys as prod_desc doesn’t depend on any set of attributes (not containing prod_desc). Hence, prod_desc is the primary key.

We have the relation:
(<u>prod_desc</u>, scanned, weight, count, ema) with FD:

- (Prod_desc) -> (weight, count, ema)
- (Prod_desc) - > (Scanned)
- (weight) -> (scanned)
- (count, ema) - > (scanned)

It is also **2NF** as all the attributes are fully functionally dependent on the primary key (as the key only has 1 attribute).

But this is **not 3NF**. There is a transitive functional dependency via (weight) and (count,ema) as shown below:

- (Scanned) depends transitively on (prod_desc) through (weight)
- (Scanned) depends transitively on (prod_desc) through (count, ema)

Also, count and ema exist only if scanned is false and weight only exists if scanned is true, else they are null. Depending on the value of scanned at least one of weight, count and ema are null for all tuples.

Keeping 1 database for all attributes has high **Insert Anomaly.**

We normalise this into 3NF by also taking into consideration the fact that depending on the value of the boolean scanned, only one of (weight) or (count,ema) can exist as non null. Hence, we can split the relation into 2 relations, one that stores all tuples with scanned false and the other with scanned true. Due to the FD (prod_desc) -> (scanned), each prod_desc can only be associated with one scanned value and hence it will belong to either of the 2 relations, never both.
Since all tuples in each of the 2 relations have the same scanned value, it is implicitly implied and hence can be dropped.
Such a split removes both the transitive functional dependency as well as the insert anomaly.

Normalised 3NF DB with **No Insert Anomaly** is:

- R_1 = (<u>prod_desc</u>, count, ema) with FD: (prod_desc) -> (count, ema)
    - (scanned) is implicitly false for all tuples
- R_2 = (<u>prod_desc</u>, weight) with FD: (prod_desc) -> (weight)
    - (scanned) is implicitly true for all tuples

Since one prod_desc can only belong to one of the 2 relations as shown above, finding which relation any prod_desc belongs to gives us its (scanned) value. Hence, no information or FD is lost.

For both relations, for every FD X->Y, X is (prod_desc) which is the primary key, and hence a superkey.

Hence, both relations are **BCNF**.

Choosing Databases

Consider the requirements:

1. High frequency updates to ‘count’ attribute of a specific tuple
2. Quick Full-Text Search of all tuples for closest ‘prod_desc’ and its corresponding Weight
3. Weekly Update of EMA of all tuples
4. Updation of scanned attribute on scan

Requirements 1 and 3 correspond entirely to R_1 and requirement 2 corresponds entirely to R_2.
Requirement 4 involves deletion of attribute from R_1 and insert in R_2.

Relation 1 needs to be highly optimised and scalable for rapid updates while Relation 2 needs to be highly optimised for Full-Text Search on the entire relation. Based on this requirements the appropriate databases to use for them are:

1. Relation 1: **Redis**
    a. Highly optimised for key based search and update
2. Relation 2: **Vector Database (OpenSearch)**
    a. Highly optimized for full-text search

II. Query Processing, Functional Indexing

The below query is to be run often on a table:

```
SELECT 1
FROM app_db.shipment_weight as weights
JOIN public.aggregated_prod_desc_data as agg_desc
ON weights.shipment_awb = agg_desc.awb
WHERE agg_desc.word_key = :prod_desc AND agg_desc.user_id = :userId
LIMIT 1;
```

The query is to check if any row matching the above conditions exists.

Query Plan summary for this query is:

Limit
-> **Nested Loop**
-> **Gather**
-> **Parallel Seq Scan** on aggregated_prod_desc_data agg_desc
Filter: ((word_key = prod_desc::text) AND (user_id = userId))
-> **Foreign Scan** on shipment_weight weights

Full query plan:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/func_idx_queryPlan.txt

The query executes by running 2 sequential scans on agg_desc table in parallel - 1 on word_key and other on user_id, filters out matching tuples, takes the intersection of the results of both scans. Then, it runs a nested loop join on both the resulting tuples and the weights table. On finding a matching tuple, it limits the execution and returns.

This query can be improved by changing the 2 parallel sequential scans into a single composite index scan. Hence, we can add the composite index (word_key, user_id) on the agg_desc table to improve efficiency. We place word_key first in the index as it is more selective than user_id.

But we run into an issue, postgres has a limit on the maximum size of the entries of attributes it can index and returns the following error:

ERROR: index row size 6168 exceeds b-tree version 4 maximum 2704 for index. Values larger than 1/3 of a buffer page cannot be indexed.

This occurs because certain entries in work_key attribute are quite large strings.

In order to solve this issue we make use of the functional indexing feature in SQL. In functional indexing, we can build an index on the result of a function on an attribute and whenever a search is made on the result of the same function on this attribute, SQL will figure out that there is a functional index on this

attribute and use that index to perform index scan. Internally, SQL performs this through generated columns.

Consider the hash of the work_key attribute. It will be within the size limits of postgres and the probability of collision, of the required row with a wrong row, is extremely low with a good hash. Hence, instead of searching for the word_key itself, we can build a functional index on the md5 hash (in-built function in postgres) of the word_key and search for the md5 hash of the attribute to match the md5 hash of the required product description.

Now replacing word_key in the previously discussed composite index with the md5 hash of word_key, we can build a composite functional index (md5(word_key), user_id) on the aggr_desc table.

SQL statement:

```
CREATE INDEX aggregated_prod_desc_data_word_key_md5_idx
ON public.aggregated_prod_desc_data (md5(word_key), user_id);
```

Now we also need to modify the query to search the hash of word_key to make use of the functional index. The query on this change becomes:

```
SELECT 1
FROM app_db.shipment_weight as weights
JOIN public.aggregated_prod_desc_data as agg_desc
ON weights.shipment_awb = agg_desc.awb
WHERE md5(agg_desc.word_key) = md5(:prod_desc)
        AND agg_desc.user_id = :userId
LIMIT 1;
```

We can 1 step further and completely eliminate the minute possibility of collision causing unwanted rows to return by adding a final filter condition on the word_key itself. This will both optimize the query to run an index scan on the composite functional index to find the matching rows and the in memory check if the word_key matches. This won't add any overhead as the returning tuples should be able to fit in memory but eliminate even the negligible probability of collision causing issues. The optimized query is:

```
SELECT 1
FROM app_db.shipment_weight as weights
JOIN public.aggregated_prod_desc_data as agg_desc
ON weights.shipment_awb = agg_desc.awb
WHERE md5(agg_desc.word_key) = md5(:prod_desc)
        AND agg_desc.user_id = :userId
        AND agg_desc.word_key = :prod_desc
LIMIT 1;
```

The query plan summary after indexing is:

Limit

-> **Nested Loop**

-> **Index Scan** using composite_md5_idx on aggregated_prod_desc_data agg_desc

Index Cond: ((md5(word_key) = '516eabeddad85f18d7e74112241265d0'::text) AND (user_id = userId))

Filter: (word_key = prod_desc::text)

-> **Foreign Scan** on shipment_weight weights

Full Query Plan:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/func_idx_queryPlan.txt

We can observe that the query has replaced the parallel sequential scan + gather step with a single index scan + filter step as intended.

Query Cost and Execution Time before indexing:

Cost = 90084.34

Execution Time = 2110.622 ms

Query Cost and Execution Time after indexing:

Cost = 3142.65

Execution Time = 8.184 ms

Improvement due to Indexing:

Reduction in Query Cost due to indexing = $\frac{(90084.34 - 3142.65)}{90084.34}$ = **96.51%**

Reduction in Query Execution Time due to indexing = $\frac{(2110.622 - 8.184)}{2110.622}$ = **99.61%**

III. Query Optimization, Postgres Optimizer Analysis, Bitmap Heap Scan, Indexing

Initial Query:

The query to optimize can be found at:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/query.sql

In summary, the query:

1. Joins the rider, tour and trip tables
2. Filters
    a. tours done today (tour.created_at)

    b. which are not closed (trip.status)
    c. belonging to a specific nodeId (query parameter) (tour.node_id)
3. Groups them on each rider
4. Selects many aggregate statistics for each rider

Primary Keys (indexed by default):

1. Rider.rider_id
2. Tour.tour_id
3. Trip.trip_id

Foreign Keys:

1. Tour.rider_id
2. Trip.tour_id

Pre-existing Singular Indices:

1. On Ridertable:
    a. rider_id
2. On Tour table:
    a. tour_id
    b. node_id
    c. status
    d. rider_id
    e. tour_date
3. On Trip table:
    a. trip_id
    b. tour_id
    c. status

Pre-existing composite Indexes:

1. On Tour table:
    a. combined_pendency_tour_idx
        i. node_id
        ii. tour_date
2. On trip table
    a. combined_pendency_idx_2
        i. status
        ii. is_drop_task

Relation Algebra formulation of the SQL Query + Optimization

$$
\begin{aligned}
&{}_{\alpha}G_{\lambda}(\,\sigma_{\theta}(rider \bowtie_{rider.rider\_id=tour.rider\_id \,\wedge\, tour.node\_id\,=\,:nodeId} tour \bowtie_{tour.tour\_id=trip.tour\_id} trip))\\
\equiv\ &{}_{\alpha}G_{\lambda}(\,\sigma_{\theta}(rider \bowtie_{rider.rider\_id=tour.rider\_id \,\wedge\, tour.node\_id\,=\,:nodeId} tour \bowtie_{tour.tour\_id=trip.tour\_id} trip))
\end{aligned}
$$

where,

α = *tour.rider_id, rider.rider_name, rider.profile_image_url, rider.phone*

λ = 17 aggregate operations

θ = *(:status is null or tour.status in :status)* $\wedge$ *tour.created_at::date = current_date* $\wedge$ *trip.status <> 'CLOSED'*

Note: current_date is a in-build postgres function that will be replaced with the current date on execution

Using the equivalence rule:

$$E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2 \ \equiv\ \sigma_{\theta 2}(\,E_1 \bowtie_{\theta 1} E_2\,)$$

and the equivalence rule:

$$\sigma_{\theta 1 \wedge \theta 2}(\,E_1 \bowtie_{\theta 3} E_2\,) \ \equiv\ (\sigma_{\theta 1}(\,E_1)) \bowtie_{\theta 3} (\,\sigma_{\theta 2}(E_2))$$

When $\theta 1$ *and* $\theta 2$ *involve attributes only from* E_1 *and* E_2 *respectively*

Together, we get:

$$\sigma_{\theta 1 \wedge \theta 2}(\,E_1 \bowtie_{\theta 3 \wedge \theta 4} E_2\,) \ \equiv \sigma_{\theta 1 \wedge \theta 2 \ \wedge \theta 4}(\,E_1 \bowtie_{\theta 3} E_2\,) \ \equiv\ (\sigma_{\theta 1 \wedge \theta 3}(\,E_1)) \bowtie_{\theta 3} (\,\sigma_{\theta 2}(E_2))$$

When $\theta 1,\ \theta 2$ *and* $\theta 3$ *involve attributes only from* E_1, E_2 *and* E_1 *respectively*

Applying this equivalence rule on the given query where:

1. E_1 is *tour* table
2. E_2 is *trip* table
3. $\theta 1$ is *(:status is null or tour.status in :status)* $\wedge$ *tour.created_at::date = current_date*
4. $\theta 2$ is *trip.status <> 'CLOSED'*
5. $\theta 3$ is *rider.rider_id=tour.rider_id*
6. $\theta 4$ is *tour.node_id = :nodeId*

we get the equivalent query:

$$_{\alpha}G_{\lambda}(\,rider \bowtie_{rider.rider\_id=tour.rider\_id} (\sigma_{\theta 1}(tour)) \bowtie_{tour.tour\_id=trip.tour\_id} (\sigma_{\theta 2}(trip))\,)$$

We can also observe that not all the columns of the table are used and hence we can project out only the relevant columns to reduce the size of the tables to process even further.

Applying these optimizations, we get the equivalent but more optimal relational algebra query:

$$_{\alpha}G_{\lambda}(\,rider \bowtie_{rider.rider\_id=tour.rider\_id} \Pi_{\beta 1}(\sigma_{\theta 1}(tour)) \bowtie_{tour.tour\_id=trip.tour\_id} \Pi_{\beta 2}(\sigma_{\theta 2}(trip))\,)$$

where,
α = *tour.rider_id, rider.rider_name, rider.profile_image_url, rider.phone*
λ = 17 aggregate operations
$\theta 1$ = *tour.created_at::date = current_date* $\wedge$ *tour.node_id = :nodeId* $\wedge$ *(:status is null or tour.status in :status)*
$\theta 2$ = *trip.status <> 'CLOSED'*
$\beta 1$ = rider_id, tour_id, tour_type, start_time, end_time, last_activity_time
$\beta 2$ = tour_id, trip_id, is_drop_task, trip_rating, cod_amount, is_escalated, payment_type, status, is_fake_attempt

Making the above relational algebra optimizations to the SQL query using Common Table Expressions (CTEs), we get the query:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/optQuery.sql

Analysis

On using the EXPLAIN keyword in SQL and getting the cost of each query, we get:

- Cost of Initial Query = 604.13
- Cost of Optimized Query = 604.13

Note the cost of the query varies from time to time and most of the time the cost of the optimized query, although negligible, was slightly lower than the initial query. There was practically no difference in actual execution times in any case.

Although the significant improvements were made to the SQL query, there is no difference in cost and practically no difference in the execution times.

This is because Postgres' optimizer is extremely good and any optimization in relation algebra of the query i.e structure of the SQL query, will practically make no difference in execution times.

This mean any optimization effort should be directed to either:

1. Improving the execution time of the query plan
2. Introducing domain knowledge to improve the query

Hence optimizing any postgres query should be done by first analysing the current execution using the EXPLAIN ANALYZE' statement and then making changes to the db itself, rather than making changes to the SQL.

We shall now optimize by improve the execution of the query plan.

Using EXPLAIN ANALYZE keyword in SQL, the query plan used by postgre is:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/queryPlan.txt

Query Plan summary before indexing:

GroupAggregate

-> **Sort**

Sort Key: t.rider_id, r.rider_name, r.rider_profile_url, r.phone, t.tour_type

Sort Method: quicksort Memory: 111kB

-> **Nested Loop**

-> **Nested Loop**

-> **Bitmap Heap Scan** on public.**tour** t

Recheck Cond: (t.node_id = :nodeId)

Filter: θ

-> **Bitmap Index Scan** on combined_pendency_tour_idx

Index Cond: (t.node_id = :nodeId)

-> **Index Scan** using rider_rider_id_idx on public.rider r

Index Cond: (r.rider_id = t.rider_id)

-> **Index Scan** using trip_tour_id_idx on public.**trip** t2

Index Cond: (t2.tour_id = t.tour_id)

Filter: ((t2.status)::text <> 'CLOSED'::text)

where θ is (((t.created_at)::date = CURRENT_DATE) AND ((:status::record IS NULL) OR ((t.status)::text = ANY(:status::text[]))))

Understanding Bitmap Heap Scan

Index Scan vs Sequential Scan:

Even though Index Scan is generally considered much better than Sequential Scan, there are instances where it's worse. In index scan, the index is scanned and for every tuple in the index that matches the index condition, it is individually loaded into memory via random I/O, filtered on other conditions in-memory, and then execution moves onto the next tuple in the index.

If a huge number of tuples match the condition, it will lead to a lot of random I/O seeks and block transfers. Further if there are multiple matching rows from the same block, the same block could even end up being seeked and loaded multiple times into memory, if it gets evicted from the buffer cache before execution reaches the next corresponding matching row in the same block.

On the contrary, sequential scan involves 1 seek and transfer of all the blocks. For any index scan, there will be a number of random I/O operations where any more number of random I/O operations will make it less efficient than sequential scan.

Usually the number of matching rows is minimal and leads to much lesser block transfers in index scan compared to sequential scan. But for a moderate number of rows returned by the index scan, where the inefficiency of index scan can build up, sequential scan can end up being better.

Bitmap Heap Scan:

Bitmap Heap Scan is a middle ground between Index Scan and Sequential Scan for when there are many random I/O operations in index scan to be quite inefficient but too less to go with sequential scan.

Bitmap Heap Scan first uses the index to find the matching rows but instead of loading them into memory and processing, it creates a bitmap of the matching rows. Once the index scan is done, it then sorts these locations by physical page order, allowing the database to access the heap (table) blocks in an optimal sequence, minimizing disk seeks.
It optimally loads the relevant blocks, rechecks for the index condition for all the rows in the loaded block in-memory and then filters the matching tuples further in-memory to get all the required tuples from that block.

1. **Reducing Random I/O**: By organizing the access pattern, it reduces random I/O operations which are typically a bottleneck in database performance.
2. **Reading Each Page Once**: Each table page is read exactly once, even if it contains multiple matching rows.

Query Explanation

The query executes by first doing a Bitmap Index scan on the tour table, with index condition on node_id and then further filtering on tour.created_at and tour.status in-memory.
Then it performs an Indexed Nested Loop Join on rider table and the resulting tuples from tour table i.e., for every tuple that is returned from the Bitmap Heap Scan, an index scan is done on the rider table for tuples with the same rider_id, following which these tuples are joined and returned.
These resulting joined tuples then undergo another Indexed Nested Loop Join with the tuples from trip table, after filtering out the tuples with status 'CLOSED' i.e., for every tuple returned by the previous join condition, an index scan is done to find tuples from trip table with the same tour_id, following which an in-memory filter on status is done before returning the final joined tuple.
Once all the above filtering and joins are performed, the resulting tuples are then sorted based on the (tour.rider_id, rider.rider_name, rider.rider_profile_url, rider.phone, tour.tour_type) attributes in memory via quicksort. The required group aggregates are then performed on them.

Indexing Analysis

Something very interesting can be found by analysing the query plan in the innermost Bitmap Index Scan for the Bitmap Heap Scan and the existing indexes.

Despite the existence of an index on tour.node_id, postgres decided to use the composite index combined_pendency_tour_idx which is an index on (tour.node_id, tour.tour_date). Despite the existence of a specialized index on node_id, whose entire reason of existence is to filter node_id as efficiently as possible, another index which is not as specialized as it is doing a better job at it (as decided by postgres' optimizer).

There can be 2 reasons for this:

1. **Size of index:**
    a. The size of the node_id index is 50M while the size of the composite index is 10M. A smaller index would be quicker to load, leading to better execution time.
    b. This is contrary to the expected outcome that the composite index would be larger than the singular index. This indicates that something might be wrong here.

c. The smaller size may be due to the more selective nature of the composite index leading to a wider but less deep index. The way the node_id index is structured, maybe it is leading to a deeper index.
d. But if making a less deep/smaller sized index is better to filter on node_id, shouldn't postgres then make such a smaller singular index on node_id.
e. The reason could be that the index is bloated and although it was optimal at time of creation, the new data is not indexed well and is ruining the index.
f. This can be fixed by reindexing the node_id index.
g. But Postgres' autovacuum should already be maintaining the indexes such that they aren't bloated. So as long as autovacuum is on, this shouldn't be the case.
h. Looking into the database setting, autovacuum is observed to be on and hence this shouldn't be the case.

2. **Correlation between created_at and tour_date:**
   a. We want to filter on node_id and date of creation_at. We can observe from the data that since the last few months, all entries have tour_date value as the same timestamp of created_at but at time 00:00:00. Hence date value for both these attributes will be the same for the current date (which is what we filter on).
   b. Postgres might be using this correlation to its advantage and hence using the composite index on (node_id, tour_date) was more optimal.
   c. Although the index condition is only on node_id, since the index contains both the node_id and tour_date columns along with the pointer to the rows, the in-memory filter step of Bitmap Heap Scan can be done just by loading the composite index on (node_id, tour_date) into memory and using the correlation to filter on tour_date instead of having to load all the matching rows into memory.
   d. Hence, postgres is able to use correlation between the columns to improve it's optimization.

Optimizing the query

We can optimize the Bitmap Heap Scan of the query execution by adding a composite functional index in the tour table on (node_id, CAST(created_at as date)). This will make the index scan return a much lower number of rows. Note that we need to filter tuples on the result of passing created_at through the CAST function (or :: operator). Hence we will need to make a functional index on created_at, else the index wouldn't be used.
Since the condition on tour.status attribute isn't very selective, adding this to the index wouldn't be effective and will only add extra size to the index.

SQL statement:

```
CREATE INDEX tour_node_id_created_at_func_idx ON public.tour (node_id, CAST(created_at AS date));
```

Note: Due to time restrictions for approval, this optimization is pending implementation.

IV. DB Redundancy Removal, Indexing

Redundancy Removal

The below ER Diagram is of a chat bot with tool calls. Every conversation involves multiple messages. Some of these messages are replies from a tool, in which case it will be associated with a tool call.

Fig 1: Old ER-Model

In the above database design there are 3 foreign keys involved where only 2 are necessary. Both 'Messages' and 'Tool Calls' relations having foreign keys to each other is **redundant**. This also causes the database to have a **circular dependency**. One of foreign key attributes between messages and tool calls tables can be removed for more efficient storage.
Tool_call_id in messages also leads to **insert anomaly** as all messages are not associated with a tool call, and hence removing tool_call_id attribute from messages table will remove the insert anomaly.

With ~1500 conversations everyday with 5.4 avg messages per conversation, there are ~8,100 rows being added to messages everyday. Not all of these messages are tool calls and hence the size of messages table is always more than tool calls table.

Removal of tool_call_id attribute (foreign key) from messages table will lead to huge saving in space as well as removing the insert anomaly. Removal of message_id from tool calls table will remove the redundant foreign key and the circular dependency but won't lead to as much space saving nor remove the insert anomaly.

Hence removal of tool_call_id from messages table leads to:

1. Removing Circular Dependency
2. Removing Redundant Attribute
3. Removing Insert Anomaly

The new ER Diagram on implementing this change is:

Fig 2: Improved ER-Diagram

Note: The database used is PostgreSQL. Though it allowed the circular dependency to exist in the database, SQLAlchemy, the ORM used by the code, did not allow to have a circular dependency in the models. Hence, the ORM had omitted the tool_call_id leaving all tool_call_id values in the messages table to be null.

Indexing

The most frequent query made on the database is one to find the latest active conversation:

```
SELECT *
FROM conversations
WHERE conversations.customer_id == cus_id AND
        conversations.status == 'active' AND
        conversations.start_time >= threshold_time
ORDER BY conversations.start_time DESC
LIMIT 1;
```

where threshold time is 1 hr ago

Query Plan summary before indexing:

Limit
-> **Sort**
Sort Key: start_time DESC
Sort Method: quicksort Memory: 25kB
-> **Seq Scan** on conversations
Filter: ((start_time >= '2025-04-12 15:40:13.976'::timestamp without time zone) AND ((customer_id)::text = :phoneNumber::text) AND ((status)::text = 'active'::text))

Full query plan:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/conv_preindex.txt

The query runs by performing a Sequential Scan on the entire table to filter the rows that match the given condition, sorting the returned tuples in memory and returning the 1st tuple.

Sequential Scan can be significantly optimized by changing it to Index Scan and hence this query can be optimized by adding an index.

Note that status will be ‘active’ if threshold_time is earlier than 1hr ago. It is periodically updated such that all old conversations have status closed.

This means that the status attribute itself and the where condition for it is redundant and can be removed.

Hence, we only need to check for customer_id and start_time. There is no need to check status. Due to lack of any index, it is currently executing this query via sequential scan so changing the query won’t make a difference. But, this does mean that a composite index on these 2 attributes will have the same filtering ability as indexing on all 3 attributes, but the index will be smaller.

Hence, to optimize this query we add a composite index (customer_id, start_time) on the conversations table. We keep customer_id first in the index as it is an equality check opposed to the ‘greater than or equal to’ check for start_time.

SQL statement:

```
CREATE INDEX conversations_customer_id_idx ON public.conversations (customer_id,start_time);
```

Query Plan summary after indexing:

Limit

-> **Index Scan** Backward using conversations_customer_id_start_time_idx on conversations
Index Cond: θ
Filter: ((status)::text = 'active'::text)

where, θ = (((customer_id)::text = '7013506943'::text) AND (start_time >= '2025-04-12 15:40:13.976'::timestamp without time zone))

Full query plan:
https://github.com/ChinmayPillai/CS315-DBMS/blob/main/Project%20on%20Prod%20DB/conv_postindex.txt

We can now see that the query executes using an Index Scan instead of Sequential Scan, which is much more efficient.
A very interesting observation can be made from the query plan. Post indexing, the query no longer sorts the tuples returned by the scan. This is probably because the index provides a sorted view of the start_time attribute, which enables the scan to return a sorted set of tuples.

Query Cost and Execution Time before indexing:

Cost = 3608.89
Execution Times:

1. Case 1: customer_id == cus_id finds 0 rows
    - Execution Time = 14.105 ms

2. Case 2: customer_id == cus_id finds rows, start_time >= threshold_time finds 0 rows
    - Execution Time = 12.176 ms
3. Case 3: matching row found
    - Execution Time = 12.830 ms

Average Execution Time = $\frac{14.105 + 12.176 + 12.830}{3}$ = 13.037 ms

Query Cost and Execution Time after indexing:

Cost = 8.44

Execution Times:

4. Case 1: customer_id == cus_id finds 0 rows
    - Execution Time = 1.4 ms
5. Case 2: customer_id == cus_id finds rows, start_time >= threshold_time finds 0 rows
    - Execution Time = 0.713 ms
6. Case 3: matching row found
    - Execution Time = 0.609 ms

Average Execution Time = $\frac{1.4 + 0.713 + 0.609}{3}$ = 0.907 ms

Improvement due to Indexing:

Reduction in Query Cost due to indexing = $\frac{(3608.89 - 8.44)}{3608.89}$ = **99.77%**

Reduction in Query Execution Time due to indexing = $\frac{(14.105 - 0.907)}{14.105}$ = **93.57%**

Conclusion

In the project, I have designed, normalised, indexed, optimized, analysed several databases and significantly increased my understanding of database systems, especially postgres. This was attained through 4 sub-projects. The concepts that have been explored in the project are:

ow concepts from the class has been analysed/implemented:

1. Normalization
2. Indexing, Functional Indexing and Query Optimization
3. Query Processing and Query Plan Analysis
4. Choosing Optimal Database Implementation
5. Postgres Optimizer Analysis
6. Reindexing and Postgres' Autovacuum
7. Bitmap Heap Scan

Utilising the concepts mentioned above and the inferences obtained from my analysis, 2 queries which are currently running in production for tens of thousands of people have been by 93-99%. Another query has also been optimized, whose implementation in production is pending. Similar metrics are expected for the same as well.